

Technical Environment Document:

CalcEngine Scientific Calculator API

Author: Raja SP, AWS Labs

Source: <https://github.com/aws-labs/aidlc-workflows>

Adaptation: Ricardo de Luna Galdino, EngSoft Learn

Download: [example-tech-env-scientific-calculator-api.pdf](#)

Project Technical Summary

- **Project Name:** CalcEngine
 - **Project Type:** Greenfield
 - **Primary Runtime Environment:** Cloud
 - **Cloud Provider:** AWS
 - **Target Deployment Model:** Serverless (API Gateway + Lambda)
 - **Package Manager:** uv
 - **Team Size:** 4 (2 backend developers, 1 frontend developer for docs portal, 1 QA engineer)
 - **Team Experience:** Strong Python backend experience, moderate AWS experience, no prior math library development. Team has used FastAPI and Flask professionally. Familiar with pytest. Limited CDK experience (will need examples).
-

Programming Languages

Required Languages

Language	Version	Purpose	Rationale
Python	3.12+	API service, math engine, Lambda handlers, CDK infrastructure	Team's primary language. Rich math ecosystem (mpmath, numpy, scipy). uv provides fast, reliable dependency management.
HTML/CSS/JS	ES2022+	Documentation portal (static site)	Minimal frontend for API docs. No framework needed; static generation with Jinja2 templates.

Permitted Languages

Language	Conditions for Use
Rust	Approved for performance-critical math functions (e.g., expression parser) if Python performance is insufficient. Requires profiling evidence before adoption. Exposed to Python via PyO3/maturin.
TypeScript	Approved for CDK infrastructure if the team prefers CDK in TypeScript over Python CDK. Decision must be made before construction begins, not mid-project.

Prohibited Languages

Language	Reason	Use Instead
Java	No team expertise. Adds operational complexity (JVM cold starts in Lambda).	Python
Go	No team expertise. Python covers all current requirements.	Python
C/C++	Maintenance burden for native extensions.	Rust via PyO3 if native performance is needed

Package and Environment Management

uv as the Standard Tool

uv is the **sole package and environment management tool** for this project. Do not use pip, pip-tools, poetry, pipenv, or conda.

uv Usage Standards

```
# Project initialization (already done; do not re-run)
uv init calcengine
cd calcengine

# Adding dependencies
uv add fastapi                # Add a runtime dependency
uv add uvicorn[standard]      # Add with extras
uv add --dev pytest pytest-cov # Add a development dependency
uv add --dev mypy ruff         # Add dev tooling

# Removing dependencies
uv remove requests            # Remove a dependency

# Running commands in the project environment
uv run python -m calcengine.main # Run application
uv run pytest                    # Run tests
uv run mypy src/                 # Run type checker
uv run ruff check src/           # Run linter

# Syncing environment from lockfile
uv sync                        # Install all dependencies from uv.lock
uv sync --dev                  # Include dev dependencies

# Lockfile management
# uv.lock is auto-generated. NEVER edit it manually.
# uv.lock MUST be committed to version control.
```

Dependency File Standards

File	Purpose	Committed to Git
<code>pyproject.toml</code>	Project metadata, dependency declarations, tool configuration	Yes
<code>uv.lock</code>	Deterministic lockfile with exact resolved versions	Yes
<code>.python-version</code>	Pin the Python version for the project (e.g., <code>3.12</code>)	Yes

pyproject.toml Conventions

All project configuration lives in `pyproject.toml`. Do not create separate config files for tools that support pyproject.toml configuration.

```

[project]
name = "calcengine"
version = "0.1.0"
description = "Scientific calculator REST API"
requires-python = ">=3.12"
dependencies = [
    # Runtime dependencies listed here by uv add
]

[dependency-groups]
dev = [
    # Dev dependencies listed here by uv add --dev
]

[tool.pytest.ini_options]
testpaths = ["tests"]
addopts = "-v --tb=short --strict-markers"
markers = [
    "unit: Unit tests (fast, no external dependencies)",
    "integration: Integration tests (may require services)",
    "accuracy: Mathematical accuracy validation tests",
]

[tool.mypy]
python_version = "3.12"
strict = true
warn_return_any = true
warn_unused_configs = true
disallow_untyped_defs = true

[tool.ruff]
target-version = "py312"
line-length = 100

[tool.ruff.lint]
select = ["E", "F", "W", "I", "N", "UP", "B", "A", "SIM", "TCH"]

[tool.coverage.run]
source = ["src/calcengine"]
branch = true

[tool.coverage.report]
fail_under = 90
show_missing = true

```

Frameworks and Libraries

Required Frameworks

Framework/Library	Version	Domain	Rationale
FastAPI	0.115+	REST API framework	Async support, automatic OpenAPI spec generation, Pydantic validation, strong Python typing integration.
Pydantic	2.x	Request/response validation, settings management	Type-safe data models, JSON serialization, integral to FastAPI.
uvicorn	0.30+	ASGI server	Standard production server for FastAPI. Used locally and in Lambda via Mangum.
Mangum	1.x	Lambda adapter	Wraps FastAPI ASGI app for AWS Lambda handler. Zero-config adapter.
pytest	8.x	Testing framework	Team standard. Rich plugin ecosystem.
mypy	1.x	Static type checking	Catch type errors before runtime. Strict mode enforced.
ruff	0.8+	Linting and formatting	Replaces flake8, isort, and black in a single fast tool.
structlog	24.x+	Structured JSON logging	All Lambda handlers and API endpoints must emit structured JSON logs. Configured once in a shared module.

Framework/Library	Version	Domain	Rationale
aws-cdk-lib	2.x	Infrastructure as Code	AWS deployment. Python CDK constructs for all infrastructure.

Preferred Libraries

Use these when their capability is needed. Do not add them preemptively.

Library	Purpose	Use When
mpmath	Arbitrary-precision arithmetic	Phase 2: when arbitrary-precision mode is implemented. Not needed for MVP (IEEE 754 double is sufficient).
numpy	Array operations, linear algebra	Phase 2: when matrix/vector operations are implemented. Do not use for basic arithmetic.
scipy	Statistical distributions, numerical integration	Phase 2+: when advanced statistics and calculus are implemented.
httpx	Async HTTP client	Outbound HTTP calls (e.g., currency rate fetching in Phase 3). Preferred over requests for async compatibility.
boto3	AWS SDK	Any direct AWS service interaction not handled by CDK at deploy time (e.g., DynamoDB queries, Secrets Manager reads at runtime).
pytest-cov	Test coverage reporting	Always. Included in dev dependencies from project start.
pytest-asyncio	Async test support	When testing async FastAPI endpoints or async functions.
hypothesis	Property-based testing	Mathematical function testing. Generates random inputs to find edge cases. Strongly recommended for all math modules.
freezegun	Time mocking	When testing time-dependent logic (rate limiting, token expiry, audit timestamps).

Prohibited Libraries

Library	Reason	Alternative
Flask	Project uses FastAPI. Do not mix web frameworks.	FastAPI
Django	Excessive for an API service. ORM not needed.	FastAPI + direct DynamoDB access
requests	Synchronous-only. Blocks the async event loop in FastAPI.	httpx
sympy	Too heavy for MVP scope. Pulls in large dependency tree.	Implement expression parser directly. Re-evaluate for Phase 3 symbolic computation.
pandas	Not needed. CalcEngine processes individual calculations, not dataframes.	Standard Python or numpy for array operations when needed.
SQLAlchemy	No relational database in MVP. DynamoDB is the data store.	boto3 DynamoDB resource/client
celery	Unnecessary complexity for MVP. All calculations are synchronous and fast (<50ms).	Re-evaluate in Phase 3 for batch processing. Use SQS + Lambda if async is needed earlier.
poetry / pipenv / pip-tools	Project uses uv exclusively. Do not introduce alternative package managers.	uv
black / isort / flake8	Replaced by ruff, which combines all three.	ruff

Library Approval Process

To add a library not on the required or preferred lists:

1. Open a GitHub issue titled "Dependency Request: [library-name]"
2. Include: purpose, alternatives considered, license (must be MIT, Apache 2.0, or BSD), maintenance status (last release date, open issues count), and size impact

3. Tech lead reviews and approves or rejects
 4. If approved, add via `uv add` and update this document
-

Cloud Environment

Cloud Provider

- **Primary Provider:** AWS
- **Account Structure:** Single AWS account for MVP. Separate dev/staging/prod accounts in Phase 2.
- **Regions:** `us-east-1` (primary). No disaster recovery region for MVP. Multi-region planned for Phase 2.

Service Allow List

Service	Approved Use Cases	Constraints
AWS Lambda	API request handlers, math computation	Python 3.12 runtime. Max 256MB memory for MVP (increase if profiling shows need). 30-second timeout.
Amazon API Gateway (HTTP API)	Public REST API endpoint	HTTP API type (not REST API type). Custom domain with TLS. Usage plans for rate limiting.
Amazon DynamoDB	API key storage, usage metering, rate limit counters	On-demand capacity mode. Single-table design. TTL for rate limit windows.
Amazon S3	OpenAPI spec hosting, static documentation site, Lambda deployment packages	Bucket encryption enabled. Public access blocked except for docs site bucket (CloudFront distribution).
Amazon CloudFront	CDN for documentation portal and API spec	HTTPS only. Cache static assets aggressively.
Amazon CloudWatch	Logging, metrics, alarms, dashboards	Structured JSON logs from all Lambdas. Custom metrics for calculation counts, latency percentiles, error rates.
AWS Secrets Manager	Stripe API keys, signing keys	Automatic rotation where supported. Lambda reads at cold start, caches in memory.
AWS Certificate Manager	TLS certificates for custom domain	Used with API Gateway and CloudFront.
Amazon Cognito	Developer account authentication for docs portal and API key management	User pool for developer signup/login. Not used for API call authentication (API keys for that).

Service	Approved Use Cases	Constraints
Amazon SQS	Dead-letter queue for failed async operations	Standard queue. Used for failed billing events and error capture. Not used for calculation requests in MVP.
AWS CDK	Infrastructure as Code deployment	Python CDK. All infrastructure defined in CDK. No manual console changes.
AWS CloudTrail	API audit logging	Enabled for all management events. Data events for S3 and Lambda in production.
AWS IAM	Service permissions	Least-privilege policies per Lambda function. No wildcard resource permissions.

Service Disallow List

Service	Reason	Alternative
Amazon EC2	Operational overhead. Serverless model preferred.	Lambda for compute.
Amazon ECS / Fargate	Over-engineering for MVP request/response workload.	Lambda. Re-evaluate if cold starts become a problem.
Amazon RDS / Aurora	Relational database not needed. API key and usage data fits DynamoDB.	DynamoDB.
Amazon ElastiCache / Redis	No caching layer needed for MVP. Calculations are stateless and fast.	In-memory caching within Lambda execution context if needed.
AWS Elastic Beanstalk	Does not fit IaC model.	CDK + Lambda.
Amazon Kinesis	Streaming not needed. All calculations are synchronous request/response.	SQS if async processing is needed.
AWS Step Functions	No multi-step orchestration in MVP.	Direct Lambda invocation. Re-evaluate for Phase 3 batch processing.
Amazon SNS	No pub/sub needed in MVP.	SQS for dead-letter queues.

Service Approval Process

To use a service not on the allow list:

1. Open a GitHub issue titled "AWS Service Request: [service-name]"
2. Include: use case, cost estimate (monthly), security implications, operational burden, and why an allowed service cannot meet the need
3. Tech lead reviews. Services with PII access or network exposure require additional security review.
4. If approved, add CDK construct and update this document

Preferred Technologies and Patterns

Architecture Pattern

Modular monolith deployed as serverless functions.

CalcEngine is a single Python package with internal modules (arithmetic, trigonometry, statistics, etc.), exposed through a single FastAPI application, deployed to AWS Lambda behind API Gateway. This is not a microservice architecture.

Decision	Choice	Rationale
Architecture style	Modular monolith	Small team (4 people), single domain, no independent scaling requirements per module in MVP.
Deployment model	Single Lambda function serving all API routes via Mangum	Simplicity. One deployment artifact. Cold start amortized across all endpoints.
Module boundaries	Python packages within <code>src/calculator/</code>	Clean internal boundaries without the operational cost of separate services. Can extract to separate Lambdas later if specific endpoints need different memory/timeout.

API Design Standards

- **Style:** REST over HTTPS. JSON request and response bodies.
- **Base URL:** `https://api.calculator.io/v1/`
- **Versioning:** URL path prefix (`/v1/`, `/v2/`). Major version only. Non-breaking changes do not increment version.
- **Documentation:** OpenAPI 3.1 specification auto-generated by FastAPI. Hosted at `https://docs.calculator.io`.
- **Naming Convention:** snake_case for JSON field names (Python convention). kebab-case for URL paths.
- **Content Type:** `application/json` for all requests and responses. No XML support.

Standard Request Format:

```
{
  "expression": "sin(pi/4) * 2 + sqrt(16)",
  "options": {
    "angle_mode": "radians",
    "precision": 15
  }
}
```

Standard Success Response Format:

```
{
  "result": 5.414213562373095,
  "expression": "sin(pi/4) * 2 + sqrt(16)",
  "computation_time_ms": 2.3,
  "engine_version": "0.1.0"
}
```

Standard Error Response Format:

```
{
  "error": {
    "code": "DOMAIN_ERROR",
    "message": "Cannot compute logarithm of a negative number",
    "detail": "log(-5) is undefined for real numbers",
    "parameter": "expression",
    "documentation_url": "https://docs.calcengine.io/errors/DOMAIN_ERROR"
  }
}
```

Error Codes (MVP):

Code	HTTP Status	Meaning
<code>PARSE_ERROR</code>	400	Expression could not be parsed. Malformed syntax.
<code>DOMAIN_ERROR</code>	422	Mathematically undefined (log(-1), sqrt(-1), division by zero).
<code>OVERFLOW_ERROR</code>	422	Result exceeds representable range.
<code>INVALID_PARAMETER</code>	400	Request parameter has invalid type or value.
<code>EXPRESSION_TOO_LONG</code>	400	Expression exceeds maximum allowed length.
<code>RATE_LIMIT_EXCEEDED</code>	429	API key has exceeded its rate limit.
<code>UNAUTHORIZED</code>	401	Missing or invalid API key.
<code>INTERNAL_ERROR</code>	500	Unexpected server error.

Data Patterns

- **Primary Data Store:** DynamoDB (single-table design)
- **Entities in DynamoDB:** API keys, usage counters (per key per month), rate limit windows (per key per minute)
- **Access Pattern:** All reads and writes are by primary key (API key ID). No scans. No complex queries.
- **Caching:** No external cache. Lambda reuses DynamoDB connections across warm invocations. API key validation results cached in Lambda memory for 60 seconds.
- **No relational database:** If relational queries become necessary (reporting, analytics), evaluate DynamoDB export to S3 + Athena before adding RDS.

Logging Pattern

All log output must be structured JSON via structlog. Human-readable console output for local development only.


```

import structlog

logger = structlog.get_logger()

# Standard log call
logger.info(
    "calculation_completed",
    expression=expression,
    result=result,
    computation_time_ms=elapsed,
    api_key_id=api_key_id,
)

# Error log call
logger.error(
    "calculation_failed",
    expression=expression,
    error_code="DOMAIN_ERROR",
    error_detail=str(e),
    api_key_id=api_key_id,
)

```

Required log fields for every API request:

Field	Description
<code>request_id</code>	Unique ID per request (from API Gateway or generated)
<code>api_key_id</code>	Hashed API key identifier (never log the raw key)
<code>endpoint</code>	API path called
<code>http_method</code>	GET, POST, etc.
<code>http_status</code>	Response status code
<code>duration_ms</code>	Total request processing time
<code>timestamp</code>	ISO 8601 timestamp

Security Requirements

Authentication and Authorization

- **API Call Authentication:** API key passed in `Authorization: Bearer {key}` header. API keys are 32-character random strings, stored as bcrypt hashes in DynamoDB.
- **Developer Portal Authentication:** Amazon Cognito user pool. Email + password signup with email verification.
- **Authorization Model:** Flat. All API keys have access to all endpoints. Tier-based rate limits (free, starter, professional) enforced by usage metering, not endpoint-level permissions.
- **API Key Management:** Developers create, rotate, and revoke keys through the developer portal. Maximum 3 active keys per account.

Data Protection

- **Encryption at Rest:** DynamoDB encrypted with AWS-managed KMS key. S3 buckets encrypted with SSE-S3.
- **Encryption in Transit:** TLS 1.2+ enforced on API Gateway custom domain and CloudFront distribution. No HTTP (plaintext) endpoints.
- **PII Handling:** Developer accounts store email and hashed password. No other PII collected. Mathematical expressions are not PII. Expressions are logged for debugging but not stored permanently (CloudWatch log retention: 30 days).
- **Data Classification:** API keys = Confidential. Developer emails = Internal. Mathematical expressions and results = Public.

Input Validation

- **Expression length limit:** 4,096 characters maximum. Reject longer expressions with `EXPRESSION_TOO_LONG`.
- **Expression character allowlist:** Alphanumeric, arithmetic operators (`+`, `-`, `*`, `/`, `^`, `%`), parentheses, decimal point, comma, whitespace, and recognized function names. Reject unrecognized characters.
- **No code execution:** The expression parser must never call `eval()`, `exec()`, `compile()`, or any dynamic code execution. Expressions are parsed into an AST and evaluated by the math engine.
- **Recursion depth limit:** Expression parser limits nesting depth to 100 levels. Prevents stack overflow on deeply nested expressions like `(((((...)))))`.

- **Numeric range validation:** Results that exceed IEEE 754 double-precision range return `OVERFLOW_ERROR` instead of `Infinity` or `NaN`.

Secrets Management

- **Stripe API Keys:** Stored in AWS Secrets Manager. Read by Lambda at cold start, cached in memory.
- **Cognito Client Secret:** Stored in AWS Secrets Manager.
- **Prohibited Practices:**
 - No secrets in `pyproject.toml`, source code, or `.env` files committed to Git
 - No secrets in Lambda environment variables (use Secrets Manager at runtime)
 - No AWS access keys in code (Lambda uses IAM execution roles)
 - `.env` files for local development only, listed in `.gitignore`

Dependency Security

- **Scanning:** GitHub Dependabot enabled for Python dependencies. Alerts on known vulnerabilities.
- **License Policy:** Allowed: MIT, Apache 2.0, BSD (2-clause and 3-clause), PSF, ISC. Prohibited: GPL, LGPL, AGPL, SSPL, proprietary. Check with `uv tree` before adding new dependencies.
- **Update Policy:** Critical/High CVEs patched within 7 days. Medium within 30 days. Low evaluated quarterly.

OWASP Top 10 Compliance (2021)

A01:2021 - Broken Access Control

Control	CalcEngine Implementation
Authorization enforcement	API key validated in FastAPI middleware (<code>api/middleware/auth.py</code>) on every request before the route handler executes. No endpoint is accessible without a valid key.
Default deny	API Gateway rejects requests without an <code>Authorization</code> header at the gateway level (401). Lambda handler rejects requests with invalid or revoked keys (401).
Resource ownership	Each API key is tied to a Cognito account. Developers can only list, rotate, and revoke their own keys. DynamoDB queries are scoped to the authenticated user's partition key.
Rate limiting	Per-key rate limits enforced in middleware (<code>api/middleware/rate_limit.py</code>). Free: 10,000 calls/month, 10 calls/second. Starter: 1M/month, 50/second. Professional: 10M/month, 200/second. Exceeding limits returns 429.
CORS policy	API Gateway CORS configured to allow only the documentation portal origin (<code>https://docs.calcengine.io</code>). No wildcard origins. <code>GET</code> and <code>POST</code> methods only.
Directory traversal / path manipulation	Not applicable. CalcEngine does not serve files or accept file paths as input.

A02:2021 - Cryptographic Failures

Control	CalcEngine Implementation
Data in transit	TLS 1.2+ enforced on API Gateway custom domain and CloudFront. HTTP endpoints do not exist. API Gateway configured with <code>SecurityPolicy: TLS_1_2</code> .
Data at rest	DynamoDB encrypted with AWS-managed KMS key. S3 buckets encrypted with SSE-S3. CloudWatch logs encrypted with service-managed keys.
Password/credential storage	Developer portal passwords hashed with bcrypt (Cognito-managed). API keys stored as bcrypt hashes in DynamoDB. Raw API keys are returned exactly once at creation time and never stored or logged.
Sensitive data in responses	API responses never contain API keys, account credentials, or internal identifiers. Error messages do not leak table names, ARNs, or stack traces.
Sensitive data in logs	API key IDs (hashed identifier, not the key itself) are logged. Raw API keys are never logged. Developer emails are not included in calculation logs.

A03:2021 - Injection

Control	CalcEngine Implementation
Expression injection	The expression parser builds an AST from a strict grammar. It does not use <code>eval()</code> , <code>exec()</code> , <code>compile()</code> , or any Python code execution mechanism. Only recognized tokens (numbers, operators, parentheses, whitelisted function names) are accepted. Unrecognized tokens cause a <code>PARSE_ERROR</code> (400).
Character allowlist	Expression input restricted to: digits, decimal point, arithmetic operators (<code>+ - * / ^ %</code>), parentheses, comma, whitespace, and a fixed set of function names (<code>sin</code> , <code>cos</code> , <code>tan</code> , <code>log</code> , <code>sqrt</code> , etc.). All other characters are rejected before parsing.
NoSQL injection	DynamoDB queries use the boto3 SDK with parameterized key conditions. No string concatenation of user input into query expressions. Partition keys and sort keys are set programmatically, never interpolated from request bodies.
HTTP header injection	FastAPI and Pydantic validate and type-check all request input. Response headers are set programmatically by the framework, not from user input.
Log injection	structlog escapes special characters in log values. User-supplied expressions are logged as string values within structured JSON fields, not interpolated into log format strings.

A04:2021 - Insecure Design

Control	CalcEngine Implementation
Threat modeling	Threat model created during AIDLC NFR Requirements stage. Reviewed when new endpoints or integration points are added. Primary threats: expression injection, resource exhaustion, API key abuse.
Defense in depth	Validation at three layers: (1) API Gateway request validation, (2) Pydantic model validation in FastAPI, (3) domain validation in engine functions. Each layer rejects independently.
Business logic limits	Expression length capped at 4,096 characters. Parser recursion depth capped at 100 levels. Maximum array size for statistics endpoints: 10,000 elements. These limits prevent resource exhaustion without affecting legitimate use.
Abuse case testing	Test suite includes negative/abuse tests: oversized expressions, deeply nested parentheses, expressions designed to cause slow evaluation, rapid-fire requests exceeding rate limits, invalid/expired/revoked API keys.

A05:2021 - Security Misconfiguration

Control	CalcEngine Implementation
Infrastructure as Code	All infrastructure defined in AWS CDK (Python). No manual console changes. CDK diff reviewed in pull requests before deploy.
Default credentials	No default API keys, admin accounts, or hardcoded passwords in any environment. Cognito user pool requires email verification.
Error messages	Production error responses return the CalcEngine error code, a user-friendly message, and a documentation URL. They never expose Python tracebacks, Lambda ARNs, DynamoDB table names, or internal file paths. FastAPI <code>debug=False</code> in production.
Unnecessary features	No <code>/docs</code> or <code>/redoc</code> interactive endpoints exposed in production Lambda. OpenAPI spec served only from the static documentation site. No health-check endpoints that reveal version details beyond <code>engine_version</code> .
Security headers	API Gateway responses include: <code>Strict-Transport-Security: max-age=31536000; includeSubDomains</code> , <code>X-Content-Type-Options: nosniff</code> , <code>X-Frame-Options: DENY</code> , <code>Cache-Control: no-store</code> on API responses. CloudFront adds security headers to documentation site.
Lambda configuration	Lambda functions use the minimum required memory (256MB). Timeout set to 30 seconds. Reserved concurrency configured to prevent runaway scaling. No environment variables containing secrets (Secrets Manager at runtime).

A06:2021 - Vulnerable and Outdated Components

Control	CalcEngine Implementation
Dependency scanning	GitHub Dependabot enabled. Scans <code>pyproject.toml</code> and <code>uv.lock</code> for known vulnerabilities. Alerts create GitHub issues automatically.
Patch SLA	Critical/High CVEs: patched within 7 days. Medium: 30 days. Low: evaluated quarterly.
License compliance	Allowed: MIT, Apache 2.0, BSD, PSF, ISC. Prohibited: GPL, LGPL, AGPL, SSPL, proprietary. Checked with <code>uv tree</code> before adding dependencies.
Lockfile integrity	<code>uv.lock</code> committed to Git and enforced in CI. <code>uv sync --locked</code> in CI pipeline fails if lockfile is out of date. No ad-hoc <code>uv add</code> in CI.
Minimal dependencies	Prohibited libraries list prevents bloated dependency trees (no pandas, Django, SQLAlchemy, sympy in MVP). Each new dependency requires a GitHub issue with justification.

A07:2021 - Identification and Authentication Failures

Control	CalcEngine Implementation
API key hashing	API keys are 32-character cryptographically random strings (via <code>secrets.token_urlsafe</code>). Stored as bcrypt hashes. Lookup uses a key prefix (first 8 chars, stored in plaintext) to find the record, then bcrypt verify confirms the full key.
Brute force protection	API Gateway throttling: 100 requests/second per IP across all endpoints. Failed authentication attempts (invalid key) logged with <code>api_key_prefix</code> and source IP. After 50 failed auth attempts from a single IP in 5 minutes, temporary IP block via WAF rule.
Developer portal auth	Cognito enforces: minimum 12-character password, email verification required, account lockout after 5 failed login attempts.
Key rotation	Developers can create a new key before revoking the old one (overlap period for zero-downtime rotation). Maximum 3 active keys per account prevents key hoarding.
Credential exposure	API key returned exactly once at creation (in the HTTP response body). Not stored in plaintext anywhere. Not included in emails. Not visible in the developer portal after creation.
Multi-factor authentication	Not required for MVP. Cognito MFA support is available and will be enabled as an option in Phase 2 when team/enterprise accounts are introduced.

A08:2021 - Software and Data Integrity Failures

Control	CalcEngine Implementation
CI/CD pipeline security	GitHub Actions. <code>main</code> branch protected: requires PR, at least 1 review, all CI checks passing. No direct pushes to <code>main</code> . Deploy workflow triggered only on merge to <code>main</code> .
Dependency integrity	<code>uv.lock</code> contains hashes for all dependencies. <code>uv sync --locked</code> verifies hashes on install. Lockfile changes in PRs are reviewed explicitly.
Deployment artifact integrity	Lambda deployment package built in CI from a clean <code>uv sync --locked</code> install. No local builds deployed to production. CDK deploy runs only from the CI pipeline, not from developer machines.
Deserialization safety	Pydantic v2 models parse and validate all incoming JSON. No use of <code>pickle</code> , <code>yaml.load()</code> (unsafe loader), or <code>marshal</code> . Only <code>json.loads()</code> via Pydantic's JSON parsing. Pydantic <code>model_config</code> has <code>extra = "forbid"</code> to reject unexpected fields.

A09:2021 - Security Logging and Monitoring Failures

Control	CalcEngine Implementation
Security events logged	All events below are logged as structured JSON to CloudWatch: authentication failures (invalid/expired/revoked key), rate limit exceeded (429), input validation failures (400), authorization anomalies, and all 5xx errors.
Log protection	CloudWatch logs are retained for 30 days. Log group resource policy prevents deletion by Lambda execution role. CloudTrail logs management events to a separate S3 bucket with object lock.
Alerting	CloudWatch Alarms configured for: 5xx error rate > 1% over 5 minutes, authentication failure rate > 100/minute, single API key generating > 10x its rate limit in attempts, Lambda concurrent execution > 80% of reserved concurrency. Alarms notify via SNS to on-call email/SMS.
Monitoring dashboard	CloudWatch dashboard displays: request count, error rate (4xx and 5xx), p50/p95/p99 latency, auth failure count, rate limit hit count, Lambda cold start percentage, DynamoDB consumed capacity. Reviewed weekly.

A10:2021 - Server-Side Request Forgery (SSRF)

Control	CalcEngine Implementation
Applicability	Low risk for MVP. CalcEngine does not make outbound HTTP requests based on user input. The expression parser evaluates mathematical expressions; it does not fetch URLs, resolve hostnames, or make network calls.
Outbound requests	The only outbound network calls from Lambda are: (1) DynamoDB queries via AWS SDK (endpoint determined by AWS region, not user input), (2) Secrets Manager reads at cold start (secret name hardcoded in config, not user input).
Phase 3 consideration	When currency conversion is added (Phase 3), the service will fetch exchange rates from a financial data provider. At that point: the provider URL will be an environment variable (not user input), requests will use an allowlisted hostname, and responses will be validated against an expected schema before use. This section must be updated before Phase 3 launches.
Network segmentation	Lambda functions run in the AWS-managed VPC (no customer VPC for MVP). They can only reach AWS services via public endpoints. No internal services, databases, or metadata endpoints are reachable from Lambda in this configuration.

Testing Requirements

Test Strategy Overview

Test Type	Required	Coverage Target	Tooling
Unit Tests	Yes	90% line, 80% branch	pytest + pytest-cov
Mathematical Accuracy Tests	Yes	100% of implemented functions	pytest + hypothesis
Integration Tests	Yes	All API endpoints, DynamoDB interactions	pytest + moto (AWS mocking)
Load Tests	Yes (pre-launch)	1,000 concurrent requests, p50 < 50ms	Locust
Security Tests	Yes	Input validation, injection prevention	pytest (custom) + manual OWASP review
End-to-End Tests	Conditional	Critical user journeys against deployed staging	pytest + httpx against live API

Unit Testing Standards

- **Coverage Minimum:** 90% line coverage, 80% branch coverage. Enforced by `pytest-cov` with `fail_under = 90` in `pyproject.toml`.
- **Mocking Policy:** Mock AWS services (DynamoDB, Secrets Manager) with `moto`. Mock time with `freezegun`. Do not mock internal math functions. Math functions must be tested with real computation.
- **Naming Convention:** Test files mirror source files. `src/calculator/trig.py` tested in `tests/unit/test_trig.py`. Test functions named `test_<function>_<scenario>` (e.g., `test_sin_zero_returns_zero`, `test_sin_negative_pi_returns_zero`).
- **Test Location:** Separate `tests/` directory tree. Not co-located with source.

```

tests/
  unit/
    test_arithmetic.py
    test_trig.py
    test_statistics.py
    test_expression_parser.py
    test_error_handling.py
  integration/
    test_api_evaluate.py
    test_api_trig.py
    test_api_keys.py
    test_rate_limiting.py
  accuracy/
    test_trig_accuracy.py
    test_arithmetic_accuracy.py
    test_statistics_accuracy.py
  conftest.py

```

Mathematical Accuracy Testing

This is a CalcEngine-specific testing category that does not exist in most projects.

- **Reference implementation:** Every math function must be tested against Python's `math` module, `mpmath` library (at high precision), or published mathematical tables.
- **Property-based testing with hypothesis:** Use hypothesis to generate random valid inputs and verify properties hold (e.g., `sin(x)^2 + cos(x)^2 == 1`, `log(a*b) == log(a) + log(b)`).
- **Edge cases:** Every function must have explicit tests for: zero, negative zero, very small numbers (near epsilon), very large numbers, domain boundaries (e.g., `asin(1)`, `asin(1.0000001)`), and special values (pi, e, multiples of pi/2 for trig).
- **Tolerance:** Results must match reference values within 1 ULP (unit in the last place) for basic functions. Document any functions where wider tolerance is accepted, with justification.

Example accuracy test pattern:

```

import math
import pytest
from hypothesis import given, strategies as st
from calcengine.trig import sin, cos

class TestSinAccuracy:
    """Validate sin() accuracy against math.sin and known exact values."""

    @pytest.mark.accuracy
    @pytest.mark.parametrize("input_val, expected", [
        (0, 0.0),
        (math.pi / 6, 0.5),
        (math.pi / 4, math.sqrt(2) / 2),
        (math.pi / 2, 1.0),
        (math.pi, 0.0),
        (3 * math.pi / 2, -1.0),
        (2 * math.pi, 0.0),
        (-math.pi / 2, -1.0),
    ])
    def test_sin_known_values(self, input_val: float, expected: float) -> None:
        result = sin(input_val)
        assert result == pytest.approx(expected, abs=1e-15)

    @pytest.mark.accuracy
    @given(st.floats(min_value=-1e6, max_value=1e6, allow_nan=False, allow_infinity=False))
    def test_sin_matches_stdlib(self, x: float) -> None:
        assert sin(x) == pytest.approx(math.sin(x), rel=1e-15)

    @pytest.mark.accuracy
    @given(st.floats(min_value=-1e6, max_value=1e6, allow_nan=False, allow_infinity=False))
    def test_pythagorean_identity(self, x: float) -> None:
        assert sin(x) ** 2 + cos(x) ** 2 == pytest.approx(1.0, abs=1e-14)

```

Integration Testing Standards

- **Scope:** Test full API request/response cycle through FastAPI test client. Test DynamoDB interactions with moto.
- **Environment:** Local. No deployed services needed. `moto` mocks all AWS services.
- **Data Management:** Each test creates its own DynamoDB table via moto fixture and tears down after. No shared test state.

CI/CD Testing Gates

Pipeline Stage	Required Tests	Tooling	Failure Action
Pre-commit	ruff check, ruff format --check, mypy	ruff, mypy via pre-commit hooks	Block commit
Pull Request	Unit tests, accuracy tests, integration tests, coverage check	pytest, GitHub Actions	Block merge
Pre-deploy (staging)	All PR tests + load test (100 concurrent, 60 seconds)	pytest + Locust, GitHub Actions	Block deploy
Post-deploy (production)	Smoke tests (10 representative calculations against live API)	pytest + httpx	Alert on-call. Auto-rollback if >50% failure.

Running Tests Locally

```
# Run all tests
uv run pytest

# Run only unit tests
uv run pytest tests/unit/ -m unit

# Run only accuracy tests
uv run pytest tests/accuracy/ -m accuracy

# Run with coverage report
uv run pytest --cov --cov-report=term-missing

# Run type checking
uv run mypy src/

# Run linter
uv run ruff check src/ tests/

# Run formatter check (no changes)
uv run ruff format --check src/ tests/

# Run formatter (apply changes)
uv run ruff format src/ tests/
```

Project Structure

```
calcengine/
  .github/
    workflows/
      ci.yml          # GitHub Actions: lint, type check, test on PR
      deploy.yml      # GitHub Actions: CDK deploy on merge to main
  src/
    calcengine/
      __init__.py
      main.py          # FastAPI app creation, Mangum handler
      config.py        # Settings via Pydantic BaseSettings
      api/
        __init__.py
        router.py      # Top-level API router
        endpoints/
          __init__.py
          evaluate.py   # POST /v1/evaluate (expression evaluation)
          arithmetic.py # POST /v1/arithmetic/{operation}
          trigonometry.py # POST /v1/trigonometry/{function}
          statistics.py # POST /v1/statistics/{function}
          constants.py  # GET /v1/constants/{name}
        middleware/
          __init__.py
          auth.py       # API key validation middleware
          rate_limit.py # Rate limiting middleware
          request_logging.py # Structured request/response logging
        models/
          __init__.py
          requests.py   # Pydantic request models
          responses.py  # Pydantic response models
          errors.py     # Error response models and error codes
      engine/
        __init__.py
        expression_parser.py # Tokenizer, AST builder, evaluator
        arithmetic.py        # Basic math operations
        trigonometry.py      # Trig functions with domain validation
        statistics.py        # Descriptive statistics functions
        constants.py         # Mathematical constants
        combinatorics.py     # Factorial, permutations, combinations
        logarithmic.py       # Log, ln, exp functions
        validation.py        # Input validation, domain checking
        errors.py            # Math-domain exception types
      storage/
        __init__.py
        dynamodb.py        # DynamoDB client, table operations
        api_keys.py         # API key CRUD, validation, hashing
        usage.py            # Usage metering, rate limit counters
        logging.py          # structlog configuration
    infrastructure/
      app.py               # CDK app entry point
      stacks/
```

```

__init__.py
api_stack.py                # Lambda, API Gateway, custom domain
data_stack.py              # DynamoDB tables
monitoring_stack.py        # CloudWatch dashboards, alarms
auth_stack.py              # Cognito user pool
docs_stack.py              # S3 + CloudFront for documentation site
tests/
  unit/
    test_arithmetic.py
    test_trig.py
    test_statistics.py
    test_expression_parser.py
    test_combinatorics.py
    test_logarithmic.py
    test_validation.py
    test_api_keys.py
  integration/
    test_api_evaluate.py
    test_api_arithmetic.py
    test_api_trig.py
    test_api_statistics.py
    test_api_auth.py
    test_api_rate_limiting.py
  accuracy/
    test_trig_accuracy.py
    test_arithmetic_accuracy.py
    test_statistics_accuracy.py
    test_logarithmic_accuracy.py
    test_expression_parser_accuracy.py
  conftest.py                # Shared fixtures (FastAPI test client, moto mocks)
examples/
  api-endpoint/
    README.md
    example_endpoint.py
    test_example_endpoint.py
  math-function/
    README.md
    example_function.py
    test_example_function.py
  cdk-construct/
    README.md
    example_stack.py
docs/
  static/                    # Documentation portal source (Jinja2 templates)
pyproject.toml
uv.lock
.python-version             # Contains: 3.12
.gitignore
.pre-commit-config.yaml
README.md

```

Directory Rules

Directory	Contains	Rules
<code>src/calculator/</code>	All application source code	Only Python. No config files, no tests, no docs.
<code>src/calculator/engine/</code>	Pure math functions	No AWS imports. No HTTP imports. No side effects. Pure functions only. Must be testable without any mocking.
<code>src/calculator/api/</code>	FastAPI routes, middleware, models	HTTP-layer only. Calls engine functions. Does not contain math logic.
<code>src/calculator/storage/</code>	DynamoDB access layer	All AWS data access isolated here. No business logic.
<code>infrastructure/</code>	CDK stacks	Python CDK only. No application code.
<code>tests/</code>	All tests	Mirrors <code>src/</code> structure. Separate <code>unit/</code> , <code>integration/</code> , <code>accuracy/</code> directories.
<code>examples/</code>	Template code for patterns	Working code with tests and README. Updated when standards change.

Example and Template Code

Example 1: API Endpoint Pattern

`examples/api-endpoint/README.md`:

```
# API Endpoint Pattern

## What This Demonstrates
Standard pattern for adding a new calculation endpoint to CalcEngine.
Shows: route definition, Pydantic models, engine call, error handling, logging.

## When to Use
- Adding any new calculation endpoint
- Adding any new HTTP route to the API

## When Not to Use
- Internal engine functions (see math-function example)
- Infrastructure changes (see cdk-construct example)

## Customization Guide
| Element | Customize? | Notes |
|-----|-----|-----|
| Route path and method | Yes | Follow /v1/{category}/{function} convention |
| Request/response models | Yes | Define Pydantic models specific to the endpoint |
| Engine function call | Yes | Call the appropriate engine module function |
| Error handling structure | No | Always use CalcEngineError hierarchy and error_response()
|
| Logging calls | No | Always log with request_id, api_key_id, duration_ms |
| Response envelope | No | Always return {"result": ..., "expression": ..., "computation_time_ms": ..., "engine_version": ...} |
```

```
examples/api-endpoint/example_endpoint.py :
```

```

"""Example: Standard API endpoint pattern for CalcEngine."""

import time

import structlog

from fastapi import APIRouter, Depends
from pydantic import BaseModel, Field

from calcengine.api.middleware.auth import get_api_key_id
from calcengine.api.models.errors import error_response
from calcengine.api.models.responses import CalculationResponse
from calcengine.engine.errors import CalcEngineError
from calcengine.engine.trigonometry import sin

logger = structlog.get_logger()

router = APIRouter()

class SinRequest(BaseModel):
    """Request model for sine calculation."""

    value: float = Field(..., description="Input angle")
    angle_mode: str = Field(
        default="radians",
        pattern="^(radians|degrees)$",
        description="Angle unit: 'radians' or 'degrees'",
    )

@router.post("/v1/trigonometry/sin", response_model=CalculationResponse)
async def calculate_sin(
    request: SinRequest,
    api_key_id: str = Depends(get_api_key_id),
) -> CalculationResponse | dict:
    """Calculate the sine of the given value."""
    start = time.perf_counter()

    try:
        result = sin(request.value, angle_mode=request.angle_mode)
        elapsed = (time.perf_counter() - start) * 1000

        logger.info(
            "calculation_completed",
            endpoint="/v1/trigonometry/sin",
            input_value=request.value,
            angle_mode=request.angle_mode,
            result=result,
            computation_time_ms=round(elapsed, 3),
            api_key_id=api_key_id,
        )

    return CalculationResponse(

```

```

        result=result,
        expression=f"sin({request.value})",
        computation_time_ms=round(elapsed, 3),
    )

except CalcEngineError as e:
    elapsed = (time.perf_counter() - start) * 1000
    logger.warning(
        "calculation_failed",
        endpoint="/v1/trigonometry/sin",
        input_value=request.value,
        error_code=e.code,
        error_detail=str(e),
        computation_time_ms=round(elapsed, 3),
        api_key_id=api_key_id,
    )
    return error_response(e)

```

```
examples/api-endpoint/test_example_endpoint.py:
```

```

"""Example: Standard test pattern for a CalcEngine API endpoint."""

import math

import pytest
from fastapi.testclient import TestClient

from calcengine.main import app

@pytest.fixture
def client() -> TestClient:
    """Create a test client with a mocked API key."""
    return TestClient(app)

class TestSinEndpoint:
    """Tests for POST /v1/trigonometry/sin."""

    @pytest.mark.unit
    def test_sin_zero_radians(self, client: TestClient) -> None:
        response = client.post(
            "/v1/trigonometry/sin",
            json={"value": 0, "angle_mode": "radians"},
            headers={"Authorization": "Bearer test-api-key"},
        )
        assert response.status_code == 200
        data = response.json()
        assert data["result"] == pytest.approx(0.0)
        assert "computation_time_ms" in data

    @pytest.mark.unit
    def test_sin_pi_over_2_radians(self, client: TestClient) -> None:
        response = client.post(
            "/v1/trigonometry/sin",
            json={"value": math.pi / 2, "angle_mode": "radians"},
            headers={"Authorization": "Bearer test-api-key"},
        )
        assert response.status_code == 200
        assert response.json()["result"] == pytest.approx(1.0)

    @pytest.mark.unit
    def test_sin_90_degrees(self, client: TestClient) -> None:
        response = client.post(
            "/v1/trigonometry/sin",
            json={"value": 90, "angle_mode": "degrees"},
            headers={"Authorization": "Bearer test-api-key"},
        )
        assert response.status_code == 200
        assert response.json()["result"] == pytest.approx(1.0)

    @pytest.mark.unit
    def test_sin_invalid_angle_mode(self, client: TestClient) -> None:

```



```

        response = client.post(
            "/v1/trigonometry/sin",
            json={"value": 1.0, "angle_mode": "gradians"},
            headers={"Authorization": "Bearer test-api-key"},
        )
        assert response.status_code == 422 # Pydantic validation error

@pytest.mark.unit
def test_sin_missing_auth(self, client: TestClient) -> None:
    response = client.post(
        "/v1/trigonometry/sin",
        json={"value": 0},
    )
    assert response.status_code == 401

```

Example 2: Pure Math Function Pattern

`examples/math-function/README.md`:

```

# Math Function Pattern

## What This Demonstrates
Standard pattern for implementing a pure math function in the engine layer.
Shows: function signature, type hints, domain validation, error raising, docstring format.

## When to Use
- Adding any new mathematical function to src/calculator/engine/

## When Not to Use
- API endpoints (see api-endpoint example)
- Functions that require AWS or HTTP access (those belong in api/ or storage/)

## Key Rules
- No imports from api/, storage/, or any external service
- Pure functions only: same input always produces same output
- Raise CalcEngineError subclasses for domain errors, never return None or NaN
- Type hints on all parameters and return values

```

`examples/math-function/example_function.py`:

```

"""Example: Standard pattern for a pure math function in CalcEngine engine layer."""

import math

from calcengine.engine.errors import DomainError

def log_base(value: float, base: float = 10.0) -> float:
    """Compute the logarithm of a value with the given base.

    Args:
        value: The number to compute the logarithm of. Must be positive.
        base: The logarithm base. Must be positive and not equal to 1.
            Defaults to 10 (common logarithm).

    Returns:
        The logarithm of value in the given base.

    Raises:
        DomainError: If value <= 0, base <= 0, or base == 1.
    """
    if value <= 0:
        raise DomainError(
            code="DOMAIN_ERROR",
            message=f"Cannot compute logarithm of {value}",
            detail="Logarithm is only defined for positive numbers",
            parameter="value",
        )

    if base <= 0:
        raise DomainError(
            code="DOMAIN_ERROR",
            message=f"Cannot use {base} as logarithm base",
            detail="Logarithm base must be positive",
            parameter="base",
        )

    if base == 1.0:
        raise DomainError(
            code="DOMAIN_ERROR",
            message="Cannot use 1 as logarithm base",
            detail="Logarithm base 1 is undefined (division by zero in change-of-base)",
            parameter="base",
        )

    return math.log(value) / math.log(base)

```

```
examples/math-function/test_example_function.py :
```

```

"""Example: Standard test pattern for a pure math function."""

import math

import pytest
from hypothesis import given, strategies as st

from calcengine.engine.errors import DomainError
from calcengine.engine.logarithmic import log_base

class TestLogBase:
    """Tests for log_base function."""

    # --- Known values ---

    @pytest.mark.unit
    def test_log10_of_100(self) -> None:
        assert log_base(100, 10) == pytest.approx(2.0)

    @pytest.mark.unit
    def test_log2_of_8(self) -> None:
        assert log_base(8, 2) == pytest.approx(3.0)

    @pytest.mark.unit
    def test_ln_of_e(self) -> None:
        assert log_base(math.e, math.e) == pytest.approx(1.0)

    @pytest.mark.unit
    def test_log_of_1_any_base(self) -> None:
        assert log_base(1, 10) == pytest.approx(0.0)
        assert log_base(1, 2) == pytest.approx(0.0)
        assert log_base(1, math.e) == pytest.approx(0.0)

    # --- Default base ---

    @pytest.mark.unit
    def test_default_base_is_10(self) -> None:
        assert log_base(1000) == pytest.approx(3.0)

    # --- Domain errors ---

    @pytest.mark.unit
    def test_log_of_zero_raises_domain_error(self) -> None:
        with pytest.raises(DomainError, match="Cannot compute logarithm"):
            log_base(0)

    @pytest.mark.unit
    def test_log_of_negative_raises_domain_error(self) -> None:
        with pytest.raises(DomainError, match="Cannot compute logarithm"):
            log_base(-5)

    @pytest.mark.unit

```

```

def test_log_base_zero_raises_domain_error(self) -> None:
    with pytest.raises(DomainError, match="Cannot use 0"):
        log_base(10, 0)

@pytest.mark.unit
def test_log_base_one_raises_domain_error(self) -> None:
    with pytest.raises(DomainError, match="Cannot use 1"):
        log_base(10, 1)

@pytest.mark.unit
def test_log_base_negative_raises_domain_error(self) -> None:
    with pytest.raises(DomainError, match="Cannot use -2"):
        log_base(10, -2)

# --- Property-based: accuracy against stdlib ---

@pytest.mark.accuracy
@given(
    st.floats(min_value=1e-300, max_value=1e300, allow_nan=False, allow_infinity=False),
)
def test_log10_matches_stdlib(self, x: float) -> None:
    assert log_base(x, 10) == pytest.approx(math.log10(x), rel=1e-14)

@pytest.mark.accuracy
@given(
    st.floats(min_value=1e-300, max_value=1e300, allow_nan=False, allow_infinity=False),
)
def test_log2_matches_stdlib(self, x: float) -> None:
    assert log_base(x, 2) == pytest.approx(math.log2(x), rel=1e-14)

# --- Property-based: mathematical identity ---

@pytest.mark.accuracy
@given(
    a=st.floats(min_value=1e-100, max_value=1e100, allow_nan=False, allow_infinity=False),
    b=st.floats(min_value=1e-100, max_value=1e100, allow_nan=False, allow_infinity=False),
)
def test_log_product_identity(self, a: float, b: float) -> None:
    """log(a * b) should equal log(a) + log(b)."""
    if a * b > 0:
        assert log_base(a * b, 10) == pytest.approx(
            log_base(a, 10) + log_base(b, 10), rel=1e-10
        )

```

Example 3: CDK Construct Pattern

`examples/cdk-construct/README.md`:

```
# CDK Construct Pattern
```

```
## What This Demonstrates
```

```
Standard pattern for defining a CDK stack for CalcEngine infrastructure.
```

```
Shows: Lambda function, API Gateway integration, DynamoDB table, IAM permissions.
```

```
## When to Use
```

```
- Adding new infrastructure resources to the project
```

```
## Key Rules
```

```
- All infrastructure in infrastructure/stacks/ directory
```

```
- One stack per logical group (api, data, monitoring, auth, docs)
```

```
- Use environment variables from CDK context, never hardcode
```

```
- Least-privilege IAM: each Lambda gets only the permissions it needs
```

```
examples/cdk-construct/example_stack.py:
```

```

"""Example: Standard CDK stack pattern for CalcEngine."""

from aws_cdk import Duration, Stack
from aws_cdk import aws_apigatewayv2 as apigwv2
from aws_cdk import aws_dynamodb as dynamodb
from aws_cdk import aws_lambda as lambda_
from aws_cdk import aws_logs as logs
from aws_cdk.aws_apigatewayv2_integrations import HttpLambdaIntegration
from constructs import Construct

class ExampleApiStack(Stack):
    """Example stack showing Lambda + API Gateway + DynamoDB pattern."""

    def __init__(self, scope: Construct, construct_id: str, **kwargs) -> None:
        super().__init__(scope, construct_id, **kwargs)

        # DynamoDB table - single table design
        table = dynamodb.Table(
            self,
            "ExampleTable",
            partition_key=dynamodb.Attribute(
                name="PK", type=dynamodb.AttributeType.STRING
            ),
            sort_key=dynamodb.Attribute(
                name="SK", type=dynamodb.AttributeType.STRING
            ),
            billing_mode=dynamodb.BillingMode.PAY_PER_REQUEST,
            encryption=dynamodb.TableEncryption.AWS_MANAGED,
            point_in_time_recovery=True,
        )

        # Lambda function
        handler = lambda_.Function(
            self,
            "ExampleHandler",
            runtime=lambda_.Runtime.PYTHON_3_12,
            handler="calcengine.main.handler",
            code=lambda_.Code.from_asset("src/"),
            memory_size=256,
            timeout=Duration.seconds(30),
            environment={
                "TABLE_NAME": table.table_name,
                "LOG_LEVEL": "INFO",
            },
            log_retention=logs.RetentionDays.ONE_MONTH,
        )

        # Grant Lambda read/write access to DynamoDB (least privilege)
        table.grant_read_write_data(handler)

        # HTTP API Gateway with Lambda integration
        api = apigwv2.HttpApi(

```

```
        self,
        "ExampleHttpApi",
        api_name="calcengine-api",
        default_integration=HttpLambdaIntegration(
            "LambdaIntegration",
            handler,
        ),
    )
```

How This Document Feeds Into AI-DLC

Section	AI-DLC Stage	How It Is Used
Project Technical Summary	Workspace Detection	Greenfield classification, team context
Programming Languages	Code Generation	Python 3.12 enforced, no other languages without approval
uv Standards	Code Generation	All dependency operations use uv, pyproject.toml is single config source
Frameworks and Libraries	Code Generation, NFR Design	FastAPI + Pydantic + Mangum stack, prohibited library enforcement
Cloud Services Allow/Disallow	Infrastructure Design	Lambda + API Gateway + DynamoDB only for MVP
Architecture Pattern	Application Design	Modular monolith, module boundaries in engine/ vs api/ vs storage/
API Design Standards	Functional Design, Code Generation	Endpoint conventions, error codes, response format
Security Requirements	NFR Requirements, NFR Design	Input validation rules, no eval(), API key auth pattern
Testing Requirements	Code Generation, Build and Test	pytest + hypothesis, 90% coverage, accuracy tests mandatory
Project Structure	Code Generation	Exact directory layout and file placement rules
Example Code	Code Generation	Canonical patterns for endpoints, engine functions, CDK stacks